

Date:
Student Name:
Student ID:

AI Workshop #3

Supervised Learning : Classification and Regression

3.1. Introduction

In this workshop, we are going to learn about **classification** and **regression** of data using **Supervised Learning** techniques.

By the end of this workshop, you will have a better understanding of these topics:

- Differences between supervised and unsupervised learning
- Classification methods
- Logistic regression classifiers
- The Naïve Bayes classifier
- Confusion matrixes
- Support Vector Machines and SVM classifiers
- Linear and polynomial regression
- Single-variable and multivariable linear regressors
- Estimating housing prices using Support Vector Regressors

3.2. Supervised versus unsupervised learning

It's not hard to see from looking at the popular press that one of the hottest areas in artificial intelligence today is machine learning. Machine learning is commonly classified into supervised and unsupervised learning. Other classifications exist, but we'll discuss those later.

Supervised learning refers to the process of building a machine learning model that is based on labeled training data. In supervised learning, each example or row is a tuple consisting of input variables and a desired target variable. For example, a common dataset used in machine learning is the "Titanic" dataset. This dataset contains features to describe the passengers of the famous ship RMS Titanic. Some of the input features are:

- Passenger name
- Sex
- Cabin class
- Age
- Place of embarkment

And the target variable in this case would be whether the passenger survived or not.

Unsupervised learning refers to the process of building a machine learning model without relying on labeled training data. In some sense, it is the opposite of supervised learning. Since there are no labels available, you need to extract insights based on just the data given to you. With unsupervised learning, we are training a system where separate datapoints will potentially be separated into multiple clusters or groups. A key point to highlight is that we don't know exactly what the criteria of separation should be. Hence, an unsupervised learning algorithm needs to separate the given dataset into several groups in the best way possible.

Now that we've described one of the main ways in which machine learning approaches are classified, let's get into how we classify data.

3.3 What is Classification?

In this section, we will discuss supervised classification techniques. The classification process is a technique used to arrange data into a fixed number of categories so that it can be used effectively and efficiently.

In machine learning, classification is used to identify the category to which a new datapoint belongs. A classification model is built based on the training dataset containing datapoints and the corresponding labels. For example, let's say that we want to determine whether a given image contains a person's face or not. We would build a training dataset containing classes corresponding to two classes: face and no-face. A model would then be trained based on the available training samples. The trained model can then be used for inference.

A good classification system makes it easy to find and retrieve data. Classification is used extensively in face recognition, spam identification, recommendation engines, and so on. A good data classification algorithm will automatically generate the right criteria to separate the given data into the given number of classes.

For classification to generate decent results, a sufficiently large number of samples will be required so that it can generalize those criteria. If there is an insufficient number of samples, the algorithm will overfit to the training data. This means that it won't perform well on unknown data because it fine-tuned the model too much to fit into the patterns observed in training data. This is actually a common problem that occurs in the world of machine learning. It's a good idea to consider this factor when building various machine learning models.

3.4. Steps for Building a Classifier in Python

For building a classifier in Python, we are going to use Python 3 and Scikit-learn which is a tool for machine learning.

In this section, we are using Naive Bayes Classifier as example to illustrate how it work by using a simple 3-Step process.

Naïve Bayes Classifier (as example)

Naïve Bayes is a classification technique used to build classifier using the **Bayes theorem**.

The assumption is that the predictors are independent. In simple words, it assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature.

For building Naïve Bayes classifier we need to use the python library called scikit learn.

There are three types of Naïve Bayes models named **Gaussian**, **Multinomial** and **Bernoulli** under scikit learn package.

To build a Naïve Bayes machine learning classifier model, we need the following:

Dataset

In this workshop, we used the dataset named Breast Cancer Wisconsin Diagnostic Database. (<https://scikit-learn.org/stable/datasets/index.html#breast-cancer-wisconsin-diagnostic-database> (<https://scikit-learn.org/stable/datasets/index.html#breast-cancer-wisconsin-diagnostic-database>))

The dataset includes various information about breast cancer tumors, as well as classification labels of malignant or benign.

The dataset has 569 instances, or data, on 569 tumors and includes information on 30 attributes, or features, such as the radius of the tumor, texture, smoothness, and area. We can import this dataset from sklearn package.

Naïve Bayes Model

For building Naïve Bayes classifier, we need a Naïve Bayes model. As told earlier, there are three types of Naïve Bayes models named **Gaussian**, **Multinomial** and **Bernoulli** under scikit learn package.

Here, in the following example we are going to use the **Gaussian Naïve Bayes model**.

By using the above, we are going to build a Naïve Bayes machine learning model to use the tumor information to predict whether or not a tumor is malignant or benign.

To begin with, we need to install the sklearn module. It can be done with the help of the following command:

Step 1: Import Scikit-learn

This would be very first step for building a classifier in Python.

In this step, we will install a Python package called Scikit-learn which is one of the best machine learning modules in Python.

The following command will help us import the package:

```
In [1]: import sklearn
```

Step 2 : Import Scikit-learn's dataset

In this step, we can begin working with the dataset for our machine learning model. Here, we are going to use the **Breast Cancer Wisconsin Diagnostic Database**.

The dataset includes various information about breast cancer tumors, as well as classification labels of malignant or benign.

The dataset has 569 instances, or data, on 569 tumors and includes information on 30 attributes, or features, such as the radius of the tumor, texture, smoothness, and area.

With the help of the following command we can import the Scikit-learn's breast cancer

```
In [2]: from sklearn.datasets import load_breast_cancer
```

Now, the following command will load the dataset.

```
In [3]: data = load_breast_cancer()
```

Following is a list of important dictionary keys:



- Classification label names(target_names)
- The actual labels(target)
- The attribute/feature names(feature_names)
- The attribute (data)

Now, with the help of the following command, we can create new variables for each important set of information and assign the data. In other words, we can organize the data with the following commands:

```
In [4]: label_names = data['target_names']  
labels = data['target']  
feature_names = data['feature_names']  
features = data['data']
```

Now, to make it clearer we can print the class labels, the first data instance's label, our feature names and the feature's value with the help of the following commands:

```
In [5]: print(label_names)
```

```
['malignant' 'benign']
```



Now, the command below will show that they are mapped to binary values 0 and 1. Here 0 represents malignant (non-serious) cancer and 1 represents benign (serious) cancer.

You will receive the following output:

```
In [6]: print(labels[0])
```

```
0
```

The two commands given below will produce the feature names and feature values.

```
In [7]: print(feature_names[0])
```

```
mean radius
```

```
In [8]: print(features[0])
```

```
[1.799e+01 1.038e+01 1.228e+02 1.001e+03 1.184e-01 2.776e-01 3.001e-01  
1.471e-01 2.419e-01 7.871e-02 1.095e+00 9.053e-01 8.589e+00 1.534e+02  
6.399e-03 4.904e-02 5.373e-02 1.587e-02 3.003e-02 6.193e-03 2.538e+01  
1.733e+01 1.846e+02 2.019e+03 1.622e-01 6.656e-01 7.119e-01 2.654e-01  
4.601e-01 1.189e-01]
```



From the above output, we can see that the first data instance is a malignant tumor the radius of which is 1.7990000e+01.



Workshop 3.1 : Breast Cancer Wisconsin Diagnostic DataSet

Knowing about the dataset you are using is very important in ML and DataMining.

In this workshop, try to familiarize with the dataset you are using in this workshop.

1. Check how many record of data we have.
2. What are the **label_names**? How important this attribute is?
3. What are the list of **feature names**? How important they are in this dataset?
4. Pick out any FIVE **feature names**, check their data range.

Workshop 3.1 Answer

```
In [9]: ## Workshop 3.1 Coding Solution ##
```

Step 3 : Organizing data into sets

In this step, we will divide our data into two parts namely a **training set** and a **test set**.

Splitting the data into these sets is very important because we have to test our model on the unseen data.

To split the data into sets, sklearn has a function called the **train_test_split()** function.

With the help of the following commands, we can split the data in these sets:

```
In [10]: from sklearn.model_selection import train_test_split
```

The above command will import the `train_test_split` function from `sklearn` and the command below will split the data into training and test data.

In the example given below, we are using 40 % of the data for testing and the remaining data would be used for training the model.

```
In [11]: train, test, train_labels, test_labels = train_test_split(features, labels, te
```



Workshop 3.2 : Data Training

Division of dataset is important in Machine Learning and Data Mining problem. Normally, we divide the dataset into 1) Training Data 2) Testing Data. Sometime, we will even divided the dataset into 1) Training Data 2) Testing Data and 3) Validation Data as well.

1. What is the main purpose of dividing the dataset into Training and Testing Data?
2. Why we need to further divide the data into a separate Validation Dataset? Give an example to support your answer.
3. Do we need to randomly select Training Data and Testing Data or we can just divide part of it as Training Data and the other as Testing Data? When and why we need to do so?
4. What is the suggested proportion of Training vs Testing Dataset? Why?

Workshop 3.2 Answer

Step 4 : Building the model

In this step, we will be building our model.

We are going to use Naïve Bayes algorithm for building the model.

Following commands can be used to build the model:

```
In [12]: from sklearn.naive_bayes import GaussianNB
```



The above command will import the `GaussianNB` module. Now, the following command will help you initialize the model.

```
In [13]: gnb = GaussianNB()
```

We will train the model by fitting it to the data by using `gnb.fit()`.

```
In [14]: model = gnb.fit(train, train_labels)
```



Workshop 3.3 : Naive Bayes Model

In this section, we are using Naive Bayes Algorithm to build the classification model.

1. Describe and explain what is Naive Bayes Algorithm?
2. What is the basic Naive Bayes Algorithm?
3. How Naive Bayes Algorithm can be used in our Breast Cancer Classification problem?

Workshop 3.3 Answer

Step 5 : Evaluating the Model and Its Accuracy

In this step, we are going to evaluate the model by making predictions on our test data. Then we will find out its accuracy also.

For making predictions, we will use the **predict()** function.

The following command will help you do this:

```
In [15]: preds = gnb.predict(test)
```

```
In [16]: print (preds)
```

```
[1 0 0 1 1 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 0 0 1 1 1 0 0 1 1 0 0 1 0
 1 1 1 1 1 1 0 1 1 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 1 0 0 1 0 0 1 1 1 0 1 1 0
 1 1 0 0 0 1 1 1 0 0 1 1 0 1 0 0 1 1 0 0 0 1 1 1 0 1 1 0 0 1 0 1 1 0 1 0 0
 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 0 0
 0 1 1 0 1 0 1 1 1 1 0 1 1 0 1 1 1 0 1 0 0 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1
 0 0 1 1 0 1]
```

The above series of 0s and 1s are the predicted values for the tumor classes – malignant and benign.



Now, by comparing the two arrays namely `test_labels` and `preds`, we can find out the accuracy of our model.

We are going to use the **accuracy_score()** function to determine the accuracy.

Consider the following command for this:

```
In [17]: from sklearn.metrics import accuracy_score
```

```
In [18]: print(accuracy_score(test_labels,preds))
```

0.9517543859649122



The result shows that the NaïveBayes classifier is 95.17% accurate. Not bad, right?

That was machine learning classifier based on the **Naïve Bayse Gaussian model**. In this way, with the help of the above steps we can build our classifier in Python.



Workshop 3.4 : Comparison Naive Bayes Models

As mentioned, Python Sklearn provides THREE different types of Naive Bayes Models: **Gaussian**, **Multinomial** and **Bernoulli** under scikit learn package.

1. Explore and explain what are the major difference between these THREE Naive Bayes Models?
2. Rerun the above training by using Multinomial and Bernoulli models and compare their difference. Which one is better? Why?

Workshop 3.4 Answer

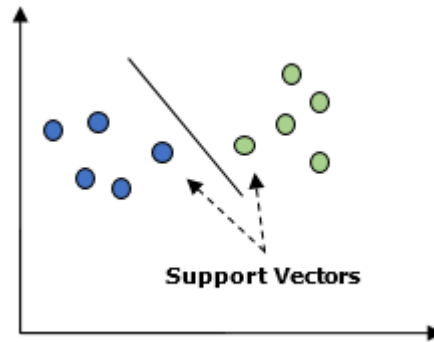
```
In [19]: ## Workshop 3.4 Coding Solution ##
```

3.5. Support Vector Machines (SVM)

Basically, Support vector machine (SVM) is a supervised machine learning algorithm that can be used for both regression and classification.

The main concept of SVM is to plot each data item as a point in n-dimensional space with the value of each feature being the value of a particular coordinate. Here n would be the features we would have.

Following is a simple graphical representation to understand the concept of SVM:

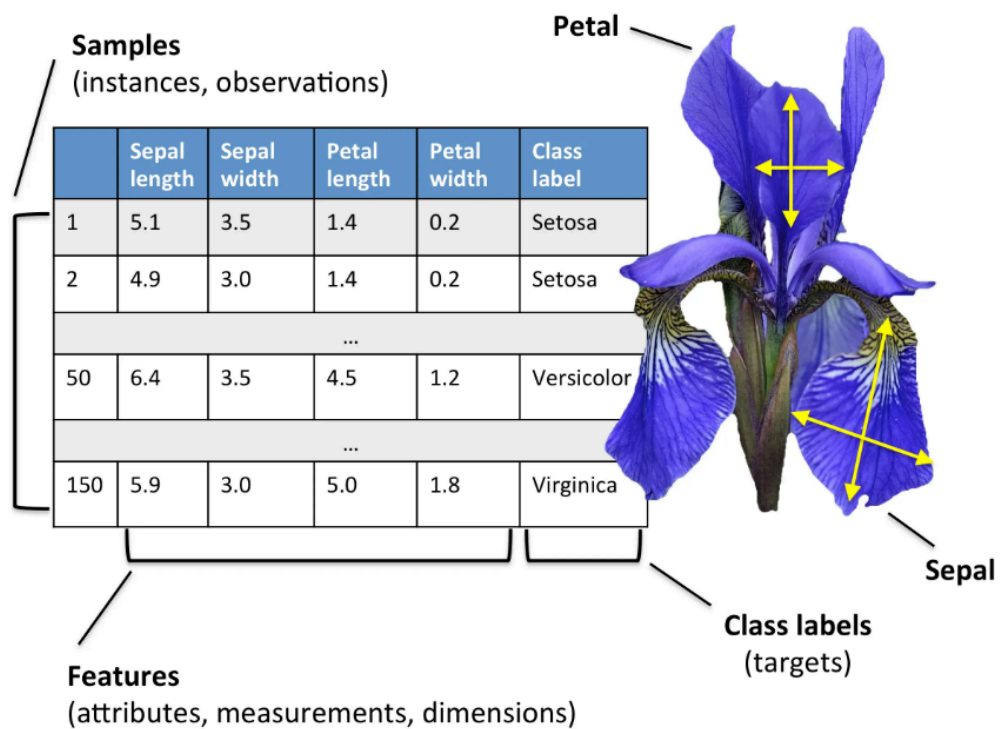


For example, in this diagram, we have two features:

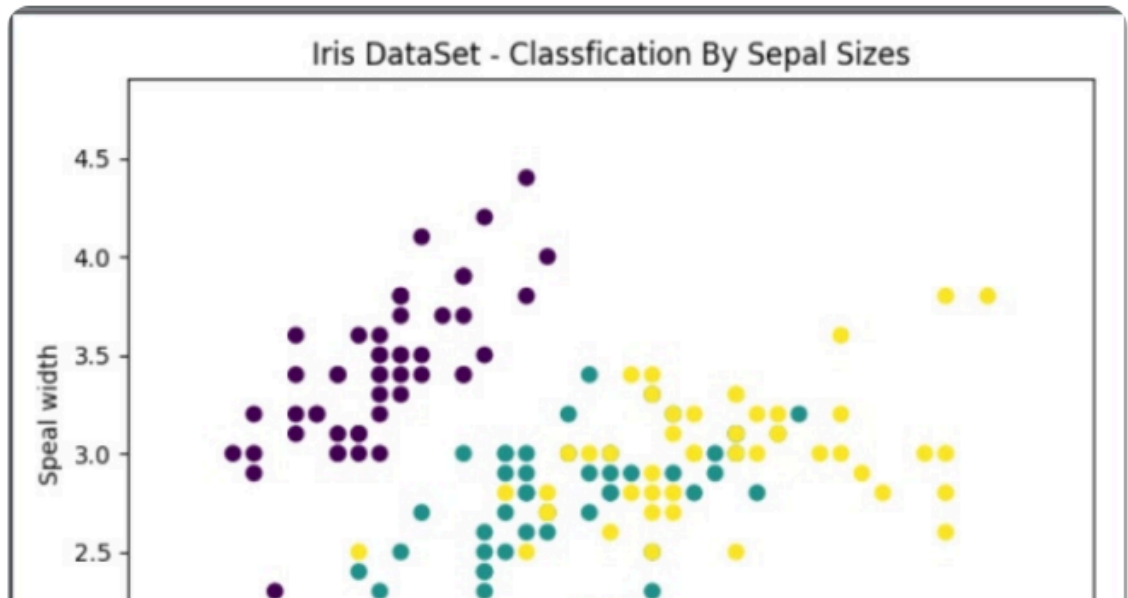
Dataset

We will use the Python built-in **iris dataset** (鸢尾花卉数据集) which contains 3 classes of 50 instances each, where each class refers to a type of iris plant.

Each instance has the four features namely sepal length, sepal width, petal length and petal width. The SVM classifier to predict the class of the iris plant based on 4 features is shown below.



Here is the data distribution:



Kernel

It is a technique used by SVM. Basically these are the functions which take low-dimensional input space and transform it to a higher dimensional space. It converts non-separable problem to separable problem.

The kernel function can be any one among linear, polynomial, rbf and sigmoid. In this example, we will use the linear kernel.

Let us now import the following packages:

```
In [20]: import pandas as pd
import numpy as np
from sklearn import svm, datasets
import matplotlib.pyplot as plt
```

Now, load the input data:

```
In [21]: iris = datasets.load_iris()
```

We are taking first two features:

```
In [22]: X = iris.data[:, :2]
y = iris.target
```

You can check with the Iris dataset:

Check the Target Dataset:

We will plot the support vector machine boundaries with original data. We are creating a mesh to plot.

```
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
h = (x_max / x_min) / 100
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
X_plot = np.c_[xx.ravel(), yy.ravel()]
```

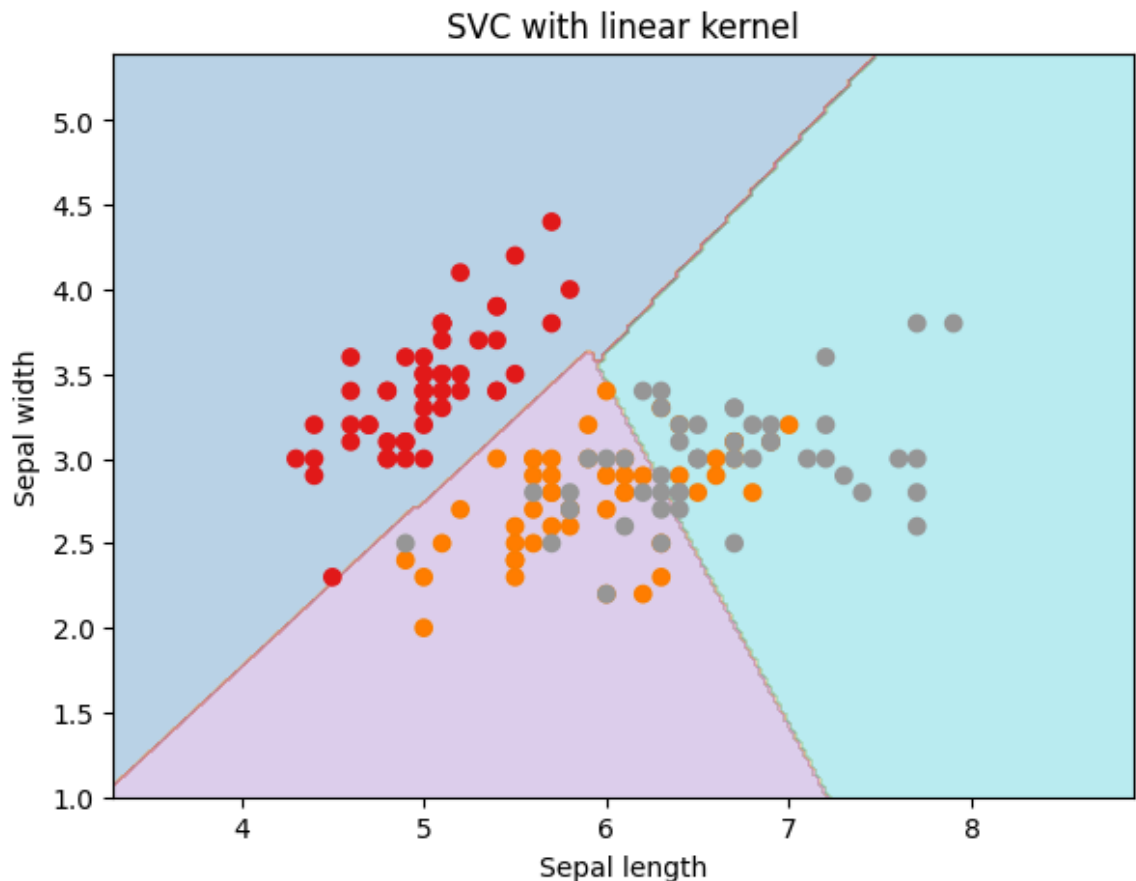
We need to give the value of regularization parameter:

C = 1.0

We need to create the SVM classifier object.

```
In [27]: svc_classifier = svm.SVC(kernel='linear',C=C, decision_function_shape = 'ovr')
Z = svc_classifier.predict(X_plot)
Z = Z.reshape(xx.shape)
plt.figure(figsize = (15, 5))
plt.subplot(121)
plt.contourf(xx, yy, Z, cmap = plt.cm.tab10, alpha = 0.3)
plt.scatter(X[:, 0], X[:, 1], c = y, cmap = plt.cm.Set1)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx.min(), xx.max())
plt.title('SVC with linear kernel')
```

Out[27]: Text(0.5, 1.0, 'SVC with linear kernel')



Workshop 3.5 : SVM on Breast Cancer Dataset

1. Based on the Breast Cancer Dataset we have obtain in the previous workshop.
2. Choose ANY two features from the dataset and use SVM method to check the classification performance. Give any comments on this.

Workshop 3.5 Answer

In [28]: `## Workshop 3.5 Coding Solution ##`

3.6 Logistic Regression

Logistic regression is a technique that is used to explain the relationship between input variables and output variables.

The input variables are assumed to be independent and the output variable is referred to as the dependent variable.

The dependent variable can take only a fixed set of values. These values correspond to the classes of the classification problem.

Our Goal

Our goal is to identify the relationship between the independent variables and the dependent variables by estimating the probabilities using a logistic function.

This logistic function is a sigmoid curve that's used to build the function with various parameters.

It is very closely related to generalized linear model analysis, where we try to fit a line to a bunch of points to minimize the error.

Instead of using linear regression, we use logistic regression. Logistic regression by itself is actually not a classification technique, but we use it in this way so as to facilitate classification.

It is used very commonly in machine learning because of its simplicity. Let's see how to build a classifier using logistic regression.

Preparation

Before building the classifier using logistic regression, we need to install the Tkinter package on our system. It can be installed from <https://docs.python.org/2/library/tkinter.html> (<https://docs.python.org/2/library/tkinter.html>) or simply using **pip install tk** command.

In [29]:

```
import tk
from tk import *
```

Let's START

First, a new Python file and import the following packages. We will be importing a function from the file **utilities.py**. We will be looking into that function very soon. But for now, let's import it:

```
In [30]: import numpy as np
from sklearn import linear_model
import matplotlib.pyplot as plt
from utilities import visualize_classifier
```

Next, define sample input data with two-dimensional vectors and corresponding labels:

```
In [31]: # Define sample input data
X = np.array([[3.1, 7.2], [4, 6.7], [2.9, 8], [5.1, 4.5], [6, 5], [5.6, 5], [
y = np.array([0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3])
```

Check the data contain in X and y. Make sure everything is okay.

```
In [32]: X
```

```
Out[32]: array([[3.1, 7.2],
               [4. , 6.7],
               [2.9, 8. ],
               [5.1, 4.5],
               [6. , 5. ],
               [5.6, 5. ],
               [3.3, 0.4],
               [3.9, 0.9],
               [2.8, 1. ],
               [0.5, 3.4],
               [1. , 4. ],
               [0.6, 4.9]])
```

```
In [33]: y
```

```
Out[33]: array([0, 0, 0, 1, 1, 1, 2, 2, 2, 3, 3, 3])
```

We will train the classifier using this labeled data. Now create the logistic regression classifier object:

```
In [34]: # Create the logistic regression classifier
classifier = linear_model.LogisticRegression(solver='liblinear', C=1)
```

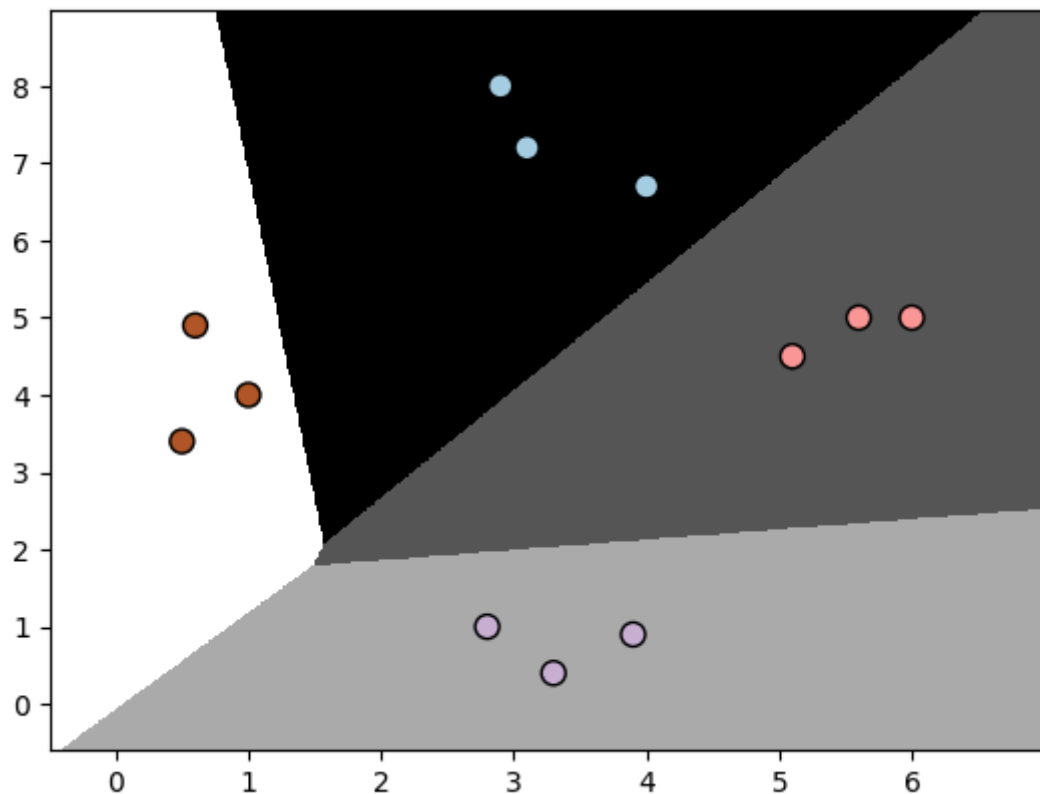
Train the classifier using the data that we defined earlier:

```
In [35]: # Train the classifier
classifier.fit(X, y)
```

```
Out[35]: LogisticRegression
LogisticRegression(C=1, solver='liblinear')
```

Visualize the performance of the classifier by looking at the boundaries of the classes by using the **visualize_classifier()** utility:

```
In [36]: # Visualize the performance of the classifier
visualize_classifier(classifier, X, y)
```



NOTE: Visualize_classifier - How it works?

We need to define this function before we can use it. We will be using this multiple times in this chapter, so it's better to define it in a separate file and import the function. This function is given in the `utilities.py` file provided to you.

The following shows you how it works.

1. Create a new Python file and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
```

2. Create the function definition by taking the classifier object, input data, and labels as input parameters:

```
def visualize_classifier(classifier, X, y):
    # Define the minimum and maximum values for X and Y
    # that will be used in the mesh grid
    min_x, max_x = X[:, 0].min() - 1.0, X[:, 0].max() + 1.0
    min_y, max_y = X[:, 1].min() - 1.0, X[:, 1].max() + 1.0
```

3. We also defined the minimum and maximum values of X and Y directions that will be used in our mesh grid. This grid is basically a set of values that is used to evaluate the

```
# Define the step size to use in plotting the mesh grid
mesh_step_size = 0.01

# Define the mesh grid of X and Y values
x_vals, y_vals = np.meshgrid(np.arange(min_x, max_x, mesh_step_size),
                             np.arange(min_y, max_y, mesh_step_size))
```

4. Run the classifier on all the points on the grid:

```
# Run the classifier on the mesh grid
output = classifier.predict(np.c_[x_vals.ravel(), y_vals.ravel()])

# Reshape the output array
output = output.reshape(x_vals.shape)
```

5. Create the figure, pick a color scheme, and overlay all the points:

```
# Create a plot
plt.figure()

# Choose a color scheme for the plot
plt.pcolormesh(x_vals, y_vals, output, cmap=plt.cm.gray)

# Overlay the training points on the plot
plt.scatter(X[:, 0], X[:, 1], c=y, s=75, edgecolors='black', linewidth=1,
            map=plt.cm.Paired)
```

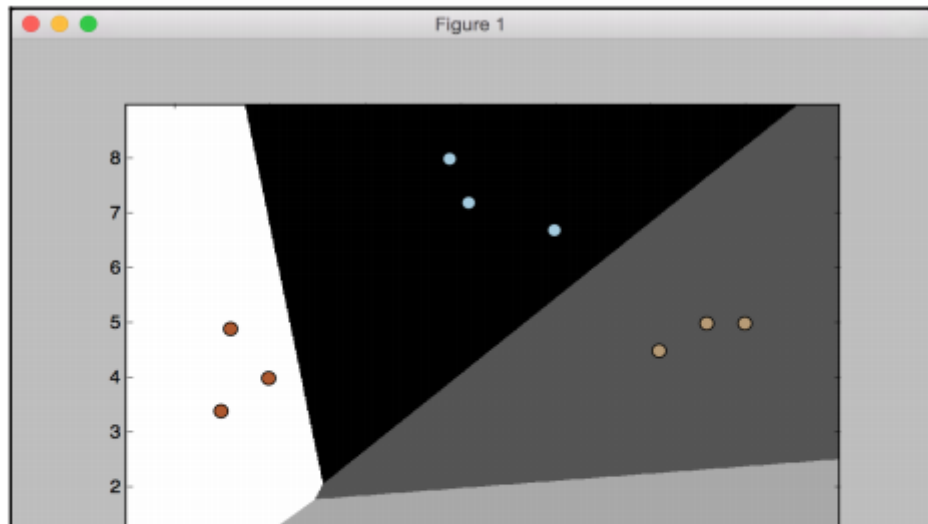
6. Specify the boundaries of the plots using the minimum and maximum values, add the tick marks, and display the figure:

```
# Specify the boundaries of the plot
plt.xlim(x_vals.min(), x_vals.max())
plt.ylim(y_vals.min(), y_vals.max())

# Specify the ticks on the X and Y axes
plt.xticks((np.arange(int(X[:, 0].min() - 1), int(X[:, 0].max() + 1), 1.0)))
plt.yticks((np.arange(int(X[:, 1].min() - 1), int(X[:, 1].max() + 1), 1.0)))
plt.show()
```

7. If you run the code, you will see the following screenshot:

Here is the data distribution:

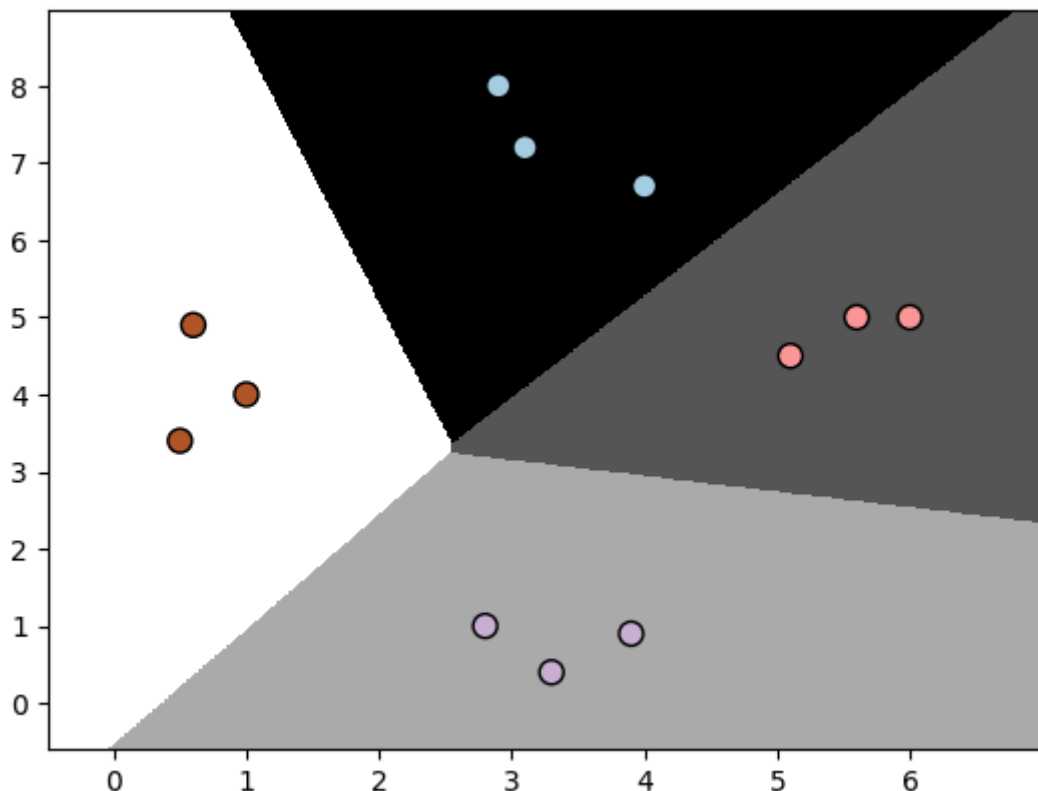


Note: If you change the value of C to 100 in the following line, you will see that the boundaries become more accurate:

```
In [37]: classifier = linear_model.LogisticRegression(solver='liblinear', C=100)

# Train the classifier
classifier.fit(X, y)

# Visualize the performance of the classifier
visualize_classifier(classifier, X, y)
```



The reason is that C imposes a certain penalty on misclassification, so the algorithm customizes more to the training data. You should be careful with this parameter, because if you increase it by a lot, it will overfit to the training data and it won't generalize well.

If you compare with the earlier figure, you will see that the boundaries are now better



Workshop 3.6 : Classification using Logistic Regression

1. By using the Iris Dataset describe in Section 3.3 as the data source and redone the classification by using Logistic Regression model.
2. Compare the results with the one using SVM. What is/are the major findings?

Workshop 3.6 Answer

In [38]: `## Workshop 3.6 Coding Solution ##`

3.7 Naïve Bayes classifier - Revisit

At the beginning of this workshop, we use a simple Naïve Bayes classifier for illustration of the basic step for Supervised Learning for classification. In this section, we revisit Naive Bayes classifier, using multivariable data with classifier visualizer to visualize the overall performance.

1. Create a new Python file and import the following packages:

```
In [39]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from utilities import visualize_classifier
```

2. We will be using the file **data_multivar_nb.txt** as the source data. This file contains comma-separated values in each line:

```
In [40]: # Input file containing data
input_file = 'data_multivar_nb.txt'
```

3. Let's load the data from this file:

```
In [41]: # Load data from input file
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

4. Take a look on the data to make sure everything okay:

```
In [42]: x
```

```
Out[42]: array([[ 2.180e+00,  5.700e-01],
 [ 4.130e+00,  5.120e+00],
 [ 9.870e+00,  1.950e+00],
 [ 4.020e+00, -8.000e-01],
 [ 1.180e+00,  1.030e+00],
 [ 4.590e+00,  5.740e+00],
 [ 8.250e+00,  1.300e+00],
 [ 3.910e+00, -6.800e-01],
 [ 5.500e-01,  1.260e+00],
 [ 5.640e+00,  6.670e+00],
 [ 9.220e+00,  1.460e+00],
 [ 4.360e+00, -1.270e+00],
 [ 2.190e+00,  2.660e+00],
 [ 4.360e+00,  4.860e+00],
 [ 7.400e+00,  4.440e+00],
 [ 3.890e+00,  1.400e-01],
 [ 1.670e+00,  8.100e-01],
 [ 4.970e+00,  5.670e+00],
 [ 8.520e+00,  2.130e+00],
 [ 5.240e+00,  7.600e-01]])
```

```
In [43]: y
```

```
Out[43]: array([0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0.,
 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1.,
 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2.,
 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3.,
 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0.,
 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1.,
 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2.,
 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3.,
 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0.,
 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1.,
 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2.,
 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3., 0., 1., 2., 3.,
 3., 0., 1., 2., 3., 0., 1., 2., 3.])
```

5. Create an instance of the Naïve Bayes classifier. We will be using the Gaussian Naïve Bayes classifier here. In this type of classifier, we assume that the values associated with each class follow a Gaussian distribution:

```
In [44]: # Create Naïve Bayes classifier
classifier = GaussianNB()
```

6. Train the classifier using the training data:

```
In [45]: # Train the classifier
classifier.fit(X, y)
```

```
Out[45]:
▼ GaussianNB
GaussianNB()
```

7. Run the classifier on the training data and predict the output:

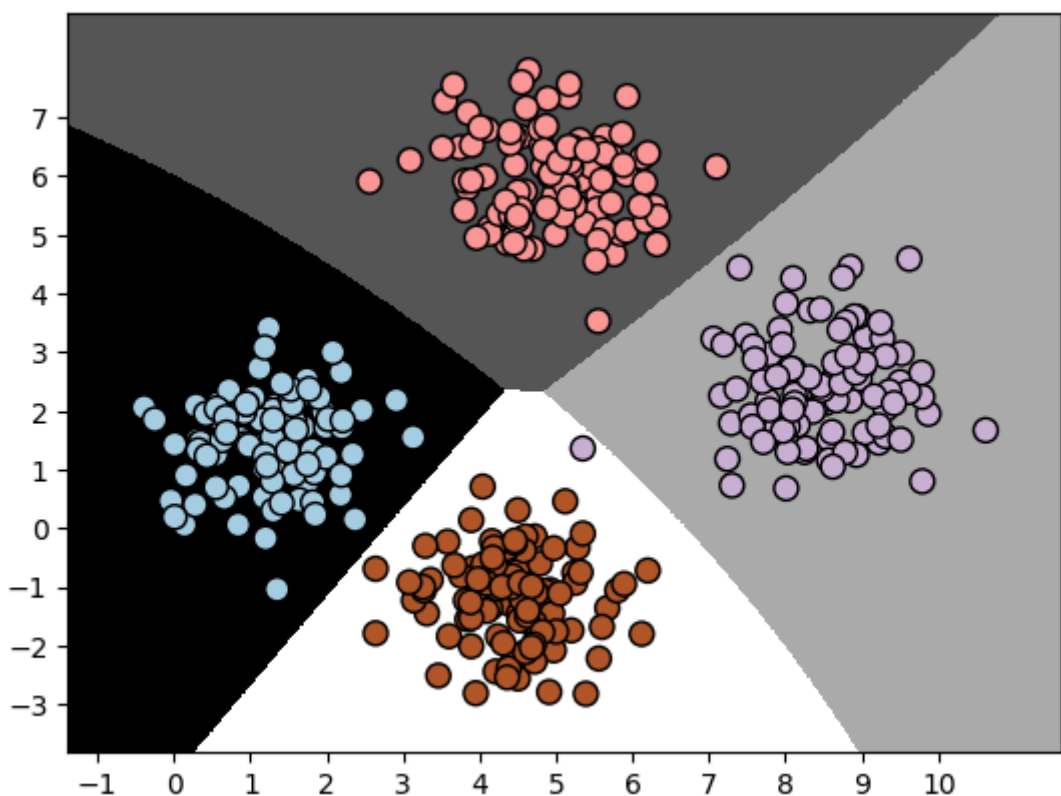
```
In [46]: # Predict the values for training data
y_pred = classifier.predict(X)
```

8. Let's compute the accuracy of the classifier by comparing the predicted values with the true labels, and then visualize the performance:

```
In [47]: # Compute accuracy
accuracy = 100.0 * (y == y_pred).sum() / X.shape[0]
print("Accuracy of Naïve Bayes classifier =", round(accuracy, 2), "%")

# Visualize the performance of the classifier
visualize_classifier(classifier, X, y)
```

Accuracy of Naïve Bayes classifier = 99.75 %



9. The preceding method to compute the accuracy of the classifier is not robust. We need to perform cross-validation, so that we don't use the same training data when we are testing it.

Split the data into training and testing subsets. As specified by the `test_size` parameter in the following line, we will allocate 80% for training and the remaining 20% for testing. We'll then train a Naïve Bayes classifier on this data:

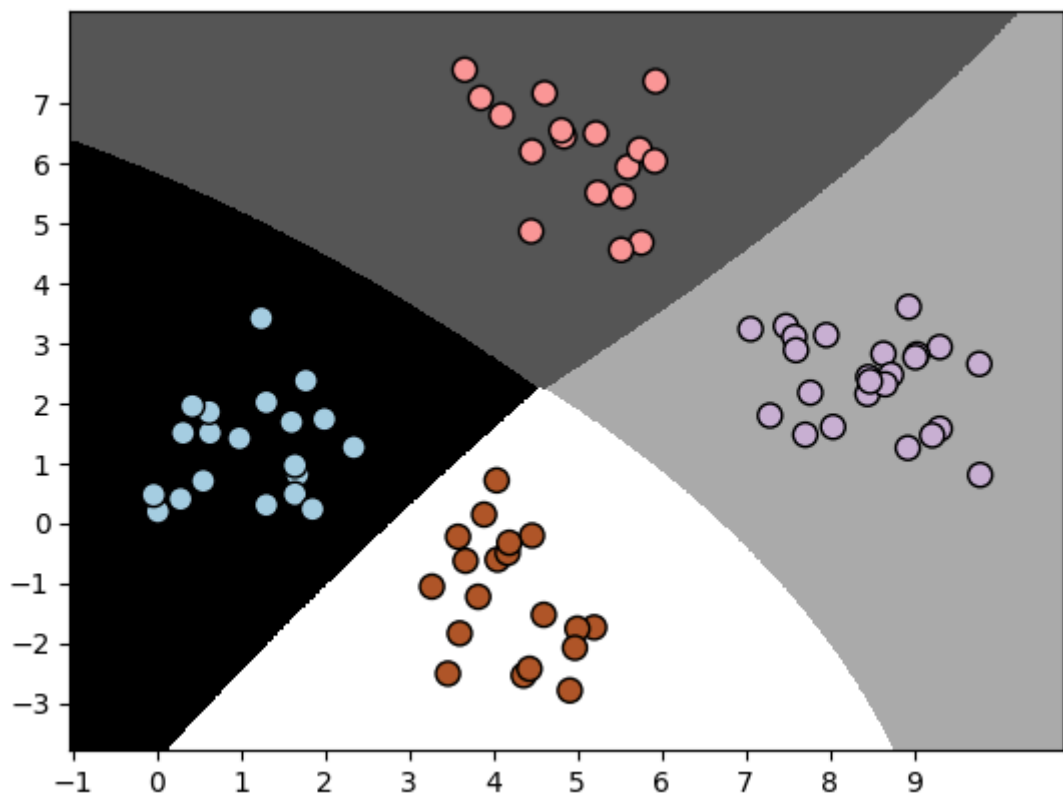
```
In [48]: # Split data into training and test data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
classifier_new = GaussianNB()
classifier_new.fit(X_train, y_train)
y_test_pred = classifier_new.predict(X_test)
```

10. Compute the accuracy of the classifier and visualize the performance:

```
In [49]: # compute accuracy of the classifier
accuracy = 100.0 * (y_test == y_test_pred).sum() / X_test.shape[0]
print("Accuracy of the new classifier =", round(accuracy, 2), "%")

# Visualize the performance of the classifier
visualize_classifier(classifier_new, X_test, y_test)
```

Accuracy of the new classifier = 100.0 %



11. Let's use the inbuilt functions to calculate the accuracy, precision, and recall values based on three-fold cross-validation:

```
In [50]: num_folds = 3
accuracy_values = cross_val_score(classifier,X, y, scoring='accuracy', cv=num_folds)
print("Accuracy: " + str(round(100*accuracy_values.mean(), 2)) + "%")
precision_values = cross_val_score(classifier,X, y, scoring='precision_weighted', cv=num_folds)
print("Precision: " + str(round(100*precision_values.mean(), 2)) + "%")
recall_values = cross_val_score(classifier,X, y, scoring='recall_weighted', cv=num_folds)
print("Recall: " + str(round(100*recall_values.mean(), 2)) + "%")
f1_values = cross_val_score(classifier,X, y, scoring='f1_weighted', cv=num_folds)
print("F1: " + str(round(100*f1_values.mean(), 2)) + "%")
```

Accuracy: 99.75%
Precision: 99.76%
Recall: 99.75%
F1: 99.75%



1. The preceding screenshot shows the boundaries obtained from the classifier. We can see that they separate the four clusters well and create regions with boundaries based on the distribution of the input datapoints as shown.
2. As shown, the overall performance is not bad.



Workshop 3.7 : Comparison with Logistic Regression and SVM

1. By using this **data_multivar_nb.txt** dataset.
2. Compare the results Logistic Regression and SVM. What is/are the major findings?

Workshop 3.7 Answer

```
In [51]: ## Workshop 3.7 Coding Solution ##
```

3.8 Confusion matrixes

A **confusion matrix** is a figure or a table that is used to describe the performance of a classifier. Each row in the matrix represents the instances in a predicted class and each column represents the instances in an actual class.

This name is used because the matrix makes it easy to visualize if the model confused or mislabeled two classes. We compare each class with every other class and see how many samples are classified correctly or misclassified.

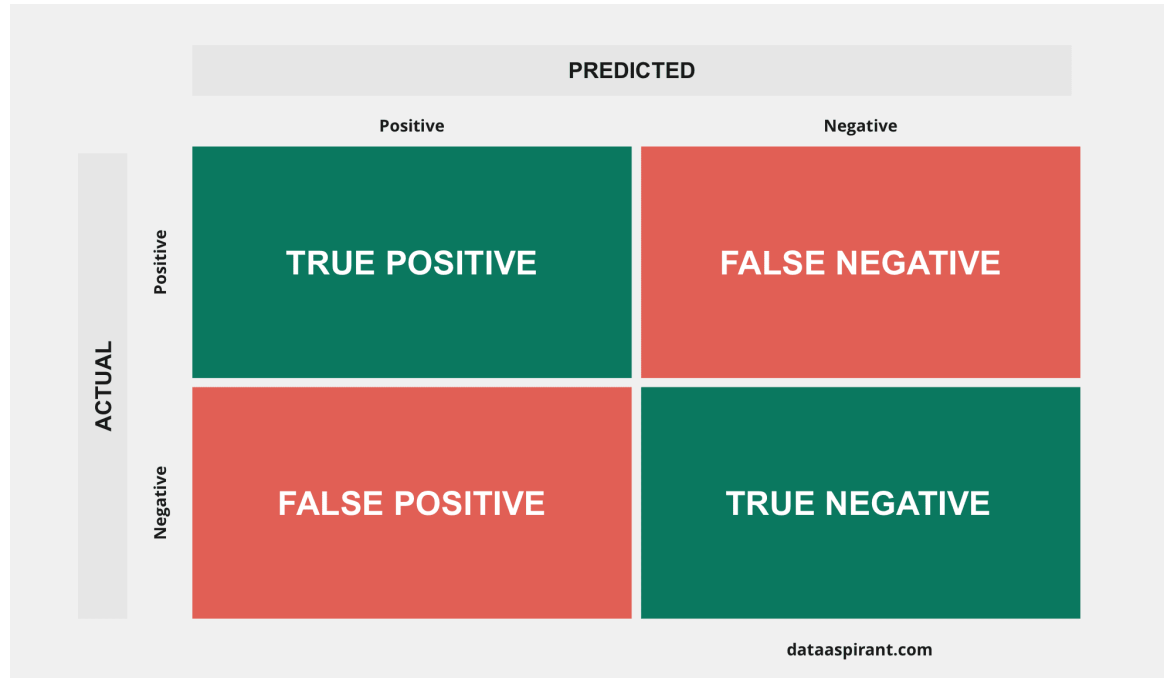
During the construction of this table, we come across several key metrics that are important in the field of machine learning.

Let's consider a binary classification case where the output is either 0 or 1:

- **True positives:** These are the samples for which we predicted 1 as the output and the ground truth is 1 too.

- **True negatives:** These are the samples for which we predicted 0 as the output and the ground truth is 0 too.
- **False positives:** These are the samples for which we predicted 1 as the output but the ground truth is 0. This is also known as a Type I error.
- **False negatives:** These are the samples for which we predicted 0 as the output but the ground truth is 1. This is also known as a Type II error.

As shown below:



Depending on the problem at hand, we may have to optimize our algorithm to reduce the false positive or the false negative rate.

For example, in biometric access system (e.g. using face or fingerprint), it is very important to avoid **false positives**, because the wrong people might get access to sensitive information.

Let's see how to create a confusion matrix.

1. Create a new Python file and import the following packages:

```
In [52]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

2. Define some sample labels for the ground truth and the predicted output:

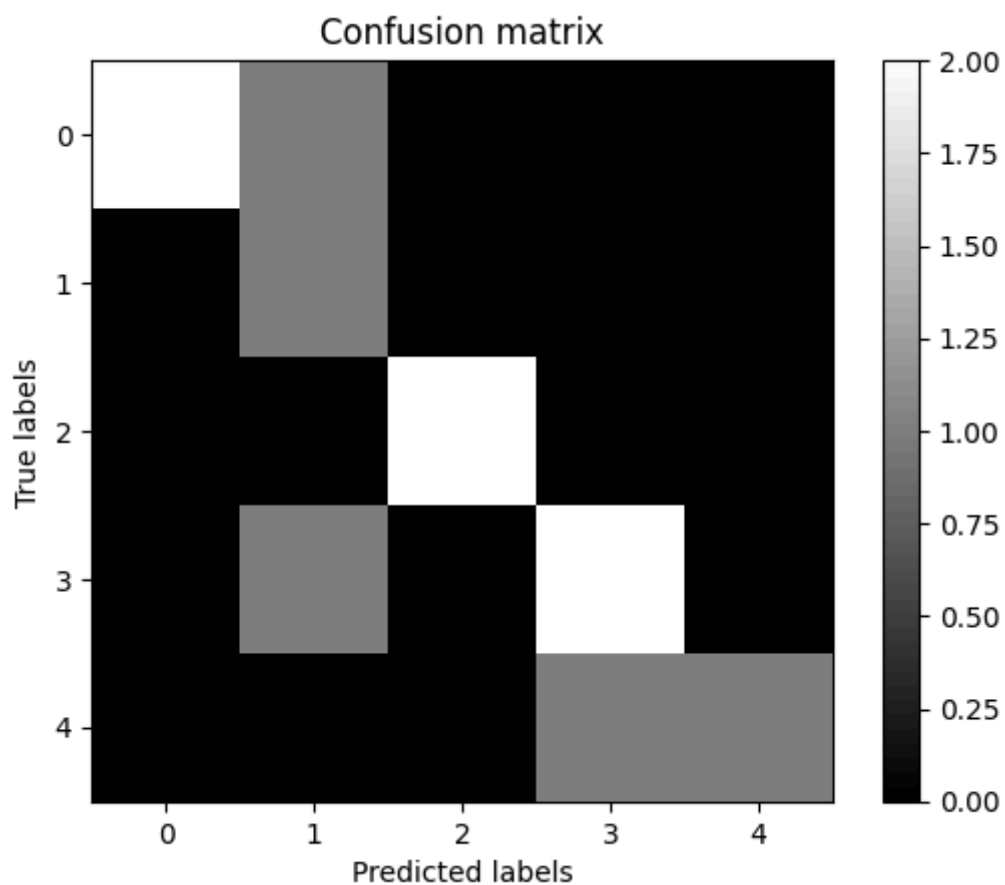
```
In [53]: # Define sample Labels
true_labels = [2, 0, 0, 2, 4, 4, 1, 0, 3, 3, 3]
pred_labels = [2, 1, 0, 2, 4, 3, 1, 0, 1, 3, 3]
```

3. Create the confusion matrix using the labels we just defined:

```
In [54]: # Create confusion matrix
confusion_mat = confusion_matrix(true_labels, pred_labels)
```

4. Visualize the confusion matrix:

```
In [55]: # Visualize confusion matrix
plt.imshow(confusion_mat, interpolation='nearest', cmap=plt.cm.gray)
plt.title('Confusion matrix')
plt.colorbar()
ticks = np.arange(5)
plt.xticks(ticks, ticks)
plt.yticks(ticks, ticks)
plt.ylabel('True labels')
plt.xlabel('Predicted labels')
plt.show()
```



5. In the preceding visualization code, the ticks variable refers to the number of distinct classes. In our case, we have five distinct labels. Let's print the classification report:

```
In [56]: # Classification report
targets = ['Class-0', 'Class-1', 'Class-2', 'Class-3', 'Class-4']
print('\n', classification_report(true_labels, pred_labels, target_names=targets))
```

	precision	recall	f1-score	support
Class-0	1.00	0.67	0.80	3
Class-1	0.33	1.00	0.50	1
Class-2	1.00	1.00	1.00	2
Class-3	0.67	0.67	0.67	3
Class-4	1.00	0.50	0.67	2
accuracy			0.73	11
macro avg	0.80	0.77	0.73	11
weighted avg	0.85	0.73	0.75	11



1. White indicates higher values, whereas black indicates lower values, as seen on the color map key on the right of the image. In an ideal scenario, the diagonal squares will be all white and everything else will be black. This indicates 100% accuracy.

2. As we can see the average precision was 85% and the average recall was 73%. Whereas the f1-score was 75%.

Depending on the domain we are working with, these might be great results or poor results. If the domain in question was to try to determine whether a patient has cancer and we were only able to have precision of 85%, 15% of the population would have been misclassified and would be quite unhappy. If the domain we were analyzing was whether someone was going to buy a product and our precision had the same results, this might be considered a home run and greatly reduce our marketing expenses.



Workshop 3.8 : Applications of Confusion Matrix

From the daily life, name ANY TWO applications of Confusion Matrix and describe how they can be used to sort out the classification problems.

Workshop 3.8 Answer

3.9 What is Regression?

Regression is the process of estimating the relationship between input and output variables. One item to note is that output variables are continuous-valued real numbers. Hence, there are an infinite number of possibilities. This is in contrast with classification, where the number of output classes is fixed. The classes belong to a finite set of possibilities.

In regression, it is assumed that the output variables depend on the input variables, so we want to see how they are related. Consequently, the input variables are called independent variables, also known as predictors, and output variables are called dependent variables, also known as criterion variables. It is not necessary that the input variables are independent of one another; indeed, there are a lot of situations where there are correlations between input variables.

Regression analysis helps us in understanding how the value of the output variable changes when we vary some input variables while keeping other input variables fixed. In linear regression, we assume that the relationship between input and output is linear. This puts a constraint on our modeling procedure, but it's fast and efficient.

Sometimes, linear regression is not enough to explain the relationship between input and output. Hence, we use polynomial regression, where we use a polynomial to explain the relationship between input and output. This is computationally more complex but gives higher accuracy. Depending on the problem at hand, we use different forms of regression to extract the relationship. Regression is frequently used for the prediction of prices, economics,

3.9.1 Single-variable Regressor

Let's see how to build a single-variable regression model.

1. Create a new Python file and import the following packages:

```
In [57]: import pickle
import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
import matplotlib.pyplot as plt
```

2. We will use the file **data_singlevar_regr.txt** provided to you. This is our source of data:

```
In [58]: # Input file containing data
input_file = 'data_singlevar_regr.txt'
```

3. It's a simple CSV file, so we can easily load it using a one-line function call:

```
In [59]: # Read data
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

4. Check X and y to make sure everything is okay:

In [60]:

x

```
Out[60]: array([[ -0.86],
 [  2.58],
 [  4.17],
 [  2.6 ],
 [  5.13],
 [  3.23],
 [ -0.26],
 [  2.76],
 [  0.47],
 [ -3.9 ],
 [  0.27],
 [  2.88],
 [ -0.54],
 [ -4.39],
 [ -1.12],
 [  2.09],
 [ -5.78],
 [  1.77],
 [ -7.91],
 [  4.86],
 [ -2.17],
 [  1.38],
 [  0.54],
 [  3.12],
 [ -2.19],
 [ -0.33],
 [ -1.21],
 [ -4.52],
 [ -0.46],
 [ -1.13],
 [  4.61],
 [  0.31],
 [  0.25],
 [ -2.67],
 [ -4.66],
 [ -0.2 ],
 [ -0.52],
 [  1.24],
 [ -2.53],
 [ -0.34],
 [  5.74],
 [ -0.34],
 [  0.99],
 [  5.01],
 [ -2.38],
 [ -0.56],
 [ -1.01],
 [  0.47],
 [  4.81],
 [  2.38],
 [ -3.32],
 [ -3.86],
 [ -1.41],
 [  6.04],
 [  4.18],
 [ -0.78],
 [ -2.2 ],
 [  0.76],
 [  2.02],
 [  0.42]])
```

In [61]: y

```
Out[61]: array([ 4.38,  6.97,  7.01,  5.44,  6.45,  5.49,  4.25,  5.94,  4.8 ,
                2.7 ,  3.26,  6.48,  4.08,  0.09,  2.74,  5.8 ,  0.16,  4.97,
               -2.26,  5.75,  3.33,  5.26,  4.43,  6.6 ,  3.77,  2.4 ,  2.98,
                0.29,  2.47,  4.08,  8.97,  3.94,  3.46,  2.46,  1.14,  4.31,
                1.97,  4.83,  3.12,  4.97,  8.65,  3.59,  3.66,  7.54,  1.52,
                4.55,  3.23,  4.39,  7.04,  4.46,  2.41,  1.11,  3.23,  7.46,
                5.71,  3.59,  2.93,  4.16,  6.43,  4.92])
```

5. Split it into training and testing:

```
In [62]: # Train and test split
num_training = int(0.8 * len(X))
num_test = len(X) - num_training
# Training data
X_train, y_train = X[:num_training], y[:num_training]
# Test data
X_test, y_test = X[num_training:], y[num_training:]
```

6. Create a linear regressor object and train it using the training data:

```
In [63]: # Create linear regressor object
regressor = linear_model.LinearRegression()

# Train the model using the training sets
regressor.fit(X_train, y_train)
```

```
Out[63]: 

LinearRegression

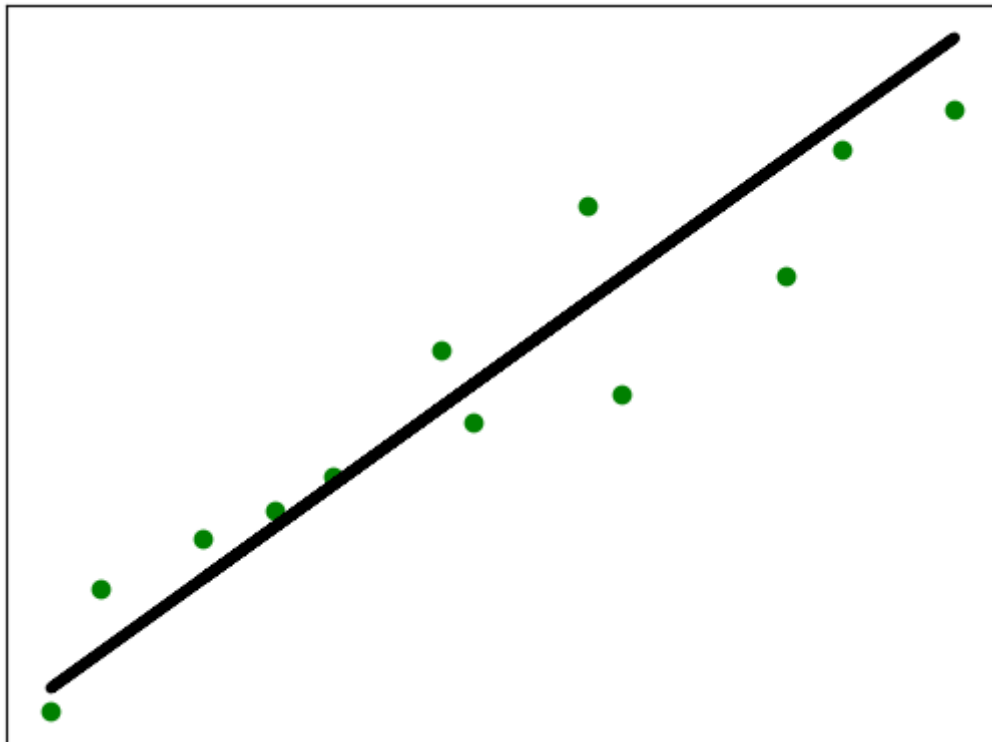

LinearRegression()
```

7. Predict the output for the testing dataset using the training model:

```
In [64]: # Predict the output
y_test_pred = regressor.predict(X_test)
```

8. Plot the output:

```
In [65]: # Plot outputs
plt.scatter(X_test, y_test, color='green')
plt.plot(X_test, y_test_pred, color='black', linewidth=4)
plt.xticks(())
plt.yticks(())
plt.show()
```



9. Compute the performance metrics for the regressor by comparing the ground truth, which refers to the actual outputs, with the predicted outputs:

```
In [66]: # Compute performance metrics
print("Linear regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test,
y_test_pred), 2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pre
print("Median absolute error =", round(sm.median_absolute_error(y_test, y_te
print("Explain variance score =", round(sm.explained_variance_score(y_test,
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))
```

```
Linear regressor performance:
Mean absolute error = 0.59
Mean squared error = 0.49
Median absolute error = 0.51
Explain variance score = 0.86
R2 score = 0.86
```

10. Once the model has been created, we can save it into a file so that we can use it later. Python provides a nice module called pickle that enables us to do this:

```
In [67]: # Model persistence
output_model_file = 'model.pkl'

# Save the model
with open(output_model_file, 'wb') as f:
    pickle.dump(regressor, f)
```

11. Let's load the model from the file on the disk and perform prediction:

```
In [68]: # Load the model
with open(output_model_file, 'rb') as f:
    regressor_model = pickle.load(f)

# Perform prediction on test data
y_test_pred_new = regressor_model.predict(X_test)
print("\nNew mean absolute error =", round(sm.mean_absolute_error(y_test, y_
```

New mean absolute error = 0.59



1. The mean squared error (MSE) is the average of the squares of the errors—that is, the average squared difference between the predicted value and the actual value. MSE is almost always strictly positive (and not zero) because of randomness. The MSE is a measure of the quality of an estimator. It is always non-negative, and the closer the value is to zero the better.
2. Explained variation measures the proportion to which a model accounts for the variation in a dataset. Often, the variation is quantified as variance; the more specific term explained variance can also be used. The rest of the total variation is the unexplained or residual variation.
3. The coefficient of determination or R2 score is used to analyze how the differences for one variable can be explained by a difference in a second variable. For example, if a woman gets pregnant, it has a high correlation with them having a child.

3.9.2 Multi-variable Regressor

In the previous section, we discussed how to build a regression model for a single variable.

In this section, we will deal with multidimensional data.

1. Create a new Python file and import the following packages:

```
In [69]: import numpy as np
from sklearn import linear_model
import sklearn.metrics as sm
from sklearn.preprocessing import PolynomialFeatures
```

2. We will use the file **data_multivar_regr.txt** provided to you.

```
In [70]: # Input file containing data
input_file = 'data_multivar_regr.txt'
```

3. This is a comma-separated file, so we can load it easily with a one-line function call:

```
In [71]: # Load the data from the input file
data = np.loadtxt(input_file, delimiter=',')
X, y = data[:, :-1], data[:, -1]
```

4. Check X and y to make sure everything is okay:

```
In [72]: X
```

```
Out[72]: array([[ 2.06,  3.48,  7.21],
 [ 6.37,  3.01,  7.27],
 [ 1.18,  1.2 ,  5.42],
 ...,
 [ 3.61,  2.22,  5.77],
 [ 1.38,  2.69, -0.33],
 [ 5.35,  5. ,  5.08]])
```

In [73]:

y

```
Out[73]: array([ 15.69, 15.34, 0.66, 38.37, 9.96, 23.4 , 24.15, 24.89,
 30.23, 41.2 , 41.35, 29.32, 22.85, 21.27, 15.88, 14.21,
 35.16, 39.08, 26.12, 3.9 , 12.64, 33.7 , 6.83, 33.47,
 36.05, 4.67, 24.09, 34.73, 35.51, 27.5 , 49.47, 27.05,
 35.34, 31.82, 18.7 , -0.38, 41.75, 41.74, 32.11, 16.86,
 14.41, 17.82, 20.97, 26.3 , 9.05, 15.33, 6.86, 32.85,
 19.93, 42.34, 3.61, 36.07, 26.98, 35.39, 34.44, 34.27,
 31.25, 19.96, 33.83, 28.35, 7.12, 25.25, 26.46, 10.66,
 20.06, 39.58, 19.96, 27.79, 23.84, -3.79, 27.42, -8. ,
 23.15, 39.55, 32.17, 11.3 , 49.68, 40.24, 49.52, 8.87,
 47.47, 14.47, 30.69, 31.27, 47.26, 32.2 , 8.83, 47.21,
 25.95, 27.29, -7.36, 32.78, 42.04, 41.51, 16.42, 6.23,
 35.7 , 21.35, 13.48, 17.21, 14.97, 27.89, 22.08, 20.5 ,
 34.06, 28.63, 31.23, -1.2 , -1.84, 17.74, 28.4 , -1.3 ,
 15.43, 8.22, 27.13, 31.82, 38.14, 8.11, -3.88, 35.34,
 16.87, 20.18, 31.74, 35.65, 4.38, 29.77, 16.55, 39.7 ,
 16.35, 18.13, 17.28, 40.54, 13. , 28.05, 5.85, 56.1 ,
 24.53, 29.7 , 24.07, 12.01, 22.89, 36.06, 19.91, 16.17,
 8.45, 31.85, 36.78, 37.03, 23.16, 35.15, 27.62, 30.44,
 24.38, -6. , 28.89, 8.72, 31.47, 31.32, 7.73, 20.38,
 33.32, 8.75, 28.76, 18.65, 14.21, 21.32, 34.13, 36.65,
 25.7 , 20.82, 28.36, 4.38, 39.46, 22.35, 6.77, 15.31,
 42.91, 33.89, 24.56, 30.53, 13.55, 26.49, 35.31, 36.88,
 21.88, 31.59, 13.86, 1.82, 36.59, 9.36, 39.66, 30.97,
 25.27, 27.69, 5.22, 13.41, 19.81, 34.97, 13.96, 9.07,
 -1.5 , 18.62, 20.36, 20.44, 3.78, 36.09, 3.59, 34.07,
 8.41, 17.5 , 29.15, 19.44, 29.56, 24.93, 26.47, 16.69,
 37.84, 40.71, 35.67, 0.06, 14.12, 7.47, 10.02, 20.58,
 30.62, 22.94, 18.5 , 10.94, 25.15, 20.05, 38.34, 18.11,
 40.5 , 15.41, 25.72, 21.12, 36.06, 26.92, 12.87, 24.74,
 48.31, -7.26, 17.47, 12.77, 35.63, 33.1 , 5.67, 17.89,
 34.05, 16.43, 30.61, 40.42, 17.35, -5.29, 29.95, -4.19,
 7.49, 29.23, 18.84, 13.04, 25.53, -12.3 , 32.11, 33.47,
 17.64, 8.35, 3.98, 30.36, 42.05, 28.31, 14.15, 17.51,
 34.25, 20.32, 31.91, 37.48, 44.17, 38.69, 29.59, 23.42,
 25.74, 16.73, 14.73, 22.48, 30.55, 14.58, -8.26, 19.28,
 22.49, 19.51, 24.1 , 12.59, 15.88, 32.53, 30.45, 26.32,
 24.83, 17.42, 24.14, 42.11, 28.93, 12.57, 22.21, 13.04,
 27.5 , 33.59, 38.83, 32.32, 40.18, 32.02, 8.22, 27.51,
 42.71, 36.27, 31.41, 25.33, 31.71, 25.14, -1.44, 6.52,
 14.44, 29.31, 48.29, 20.86, 32.25, 18.91, 5.79, 33.07,
 13.43, 34.34, 16.91, -2.9 , 37.85, 22.48, 15.51, 10.97,
 38.38, 8.26, 9.73, 18.46, 44.86, 42.94, 20.47, 16. ,
 18.98, 28. , 21.33, 25.29, 34.6 , 2.59, 5.59, 21.42,
 16.18, 40.72, 18.8 , 5.89, 13.67, 24.92, 22.54, 25.97,
 20.16, 46.13, 8.37, 5.24, 38.24, 18.17, 20.41, 24.54,
 19.39, 18.1 , 43.29, 29.89, 30.17, 18.43, 17.08, 21.56,
 20.59, 24.47, 20.62, 16.32, 22.88, 22.69, 33. , 31.96,
 30.08, 7.69, 28.57, 37.97, 1.55, 4.2 , 8.67, 19.39,
 12.42, 34.47, 25.77, 15.49, 19.22, 34.51, 3.07, 10.58,
 14.3 , 22.52, 10.37, 35.22, 25.88, 37.97, 25.64, 30.61,
 20.57, 19.81, 28.39, 23.12, 24.2 , 15.71, 5.72, 21.64,
 28.38, 24.51, 5.33, 38.15, 24.47, 15.22, 19.78, 31.06,
 11.37, 12.82, 27.17, 11.15, 17.14, 4.62, 23.61, 35.29,
 11.17, 44.49, 30.43, 16.23, 48.32, 25.66, 17.11, 24.53,
 11.8 , 46.64, 38.67, 32.02, 17.23, 32.57, 6.19, 27.27,
 -6.64, -1.81, 7.65, 3.09, 18.9 , 20.98, 30.7 , 24.1 ,
 35.28, 33.85, 35.69, 20.86, 12.39, 45.58, 25.31, 25.8 ,
 32.15, 28.05, 28.42, 20.54, 8.55, 15.25, 27.48, 44.93,
 34.79, 24.04, 29.34, 22.8 , 12.67, 6.69, 29.27, 56.05,
 15.84, 6.54, 23.74, 43.79, 13.6 , 35.51, 23.26, 18.09,
```

```

17.17, 17.56, 28.79, 17.4 , 7.9 , 13.83, 39.76, 25.19,
21.58, -6.11, 42.98, 42.55, 30.44, 37.25, 40.53, 28.39,
23.96, 28.48, 40.22, 8.27, 33.37, -1.39, 31.48, 27.74,
16.35, -1.76, 34.35, 12. , 9.21, 47.98, 14.2 , -7.07,
9.21, 13.66, 27.29, 30.71, 13.71, 46.51, 22.96, 34.19,
11.66, 6.44, 25.58, 0.19, 21.36, 25.55, 12.87, 1.4 ,
36.41, 33.92, 29.44, 35.98, 23.05, 29.07, 20.55, 23.14,
18.75, 30.72, 16.21, 31.01, 21.81, 14.36, 25.45, 44.21,
13.03, 8.23, 20.68, 35.2 , 20.15, 28.7 , 26.99, 11.3 ,
26.42, 18.99, 12.63, 22.02, 29.66, 30.36, 18.87, 15.63,
12.53, 2.82, 38.22, 35.05, 25.77, 8.01, 47.31, 20.47,
-7.8 , 23.21, 20. , 37.76, 10.68, 50.56, 9.23, 12.31,
10.84, 7.18, 7.94, 38.62, 32.24, 16.96, 11.78, 27.98,
19.51, 26.33, 30.4 , 16.55, 16.1 , 13.97, 11.99, 42.36,
27.7 , 46.43, 21.47, 42.74, 10.53, 3.32, 32.24, 14.01,
14.34, 8.67, 27.28, 35.55, 28.68, 31.55, 34.86, 28.31,
29.02, 41.31, 14.97, 29.43, 24.66, 14.14, 19.03, 23.4 ,
31.36, 3.14, 20.62, 2.06, 12.81, 8.59, 28.99, -1.21,
23.7 , 20.7 , 9.06, 43.34, 29.74, 2.02, -8.77, -0.99,
0.51, 33.91, 22.09, 26.33, 9.48, 23.51, 16.35, 16.65,
12.69, 38.45, 29.36, 25.08, 24.04, 24.15, 7.42, 23. ,
-6.79, 19.21, 21.32, 17.27, 19.77, 36.88, 6.5 , 20.65,
26.53, 23.05, 33.92, -2.08, 28.3 , 24.79, 16.94, 5.41,
20.84, 40.43, 8.06, 17.76, 38.66, 17.26, 17.45, 35.67,
46.62, 23.33, 30.41, 27.37, 18.44, -0.85, 18.78, 22.75,
38.65, 10.02, 19.13, 31.67, 27.14, 36.44, 18.06, 15.95,
5.24, 6.33, 26.66, 25.13])

```

5. Split the data into training and testing:

```

In [74]: # Split data into training and testing
num_training = int(0.8 * len(X))
num_test = len(X) - num_training

# Training data
X_train, y_train = X[:num_training], y[:num_training]

# Test data
X_test, y_test = X[num_training:], y[num_training:]

```

6. Create and train the linear regressor model:

```

In [75]: # Create the linear regressor model
linear_regressor = linear_model.LinearRegression()

# Train the model using the training sets
linear_regressor.fit(X_train, y_train)#

```

```

Out[75]:
LinearRegression
LinearRegression()

```

7. Predict the output for the test dataset:

```
In [76]: # Predict the output
y_test_pred = linear_regressor.predict(X_test)
```

8. Print the performance metrics:

```
In [77]: # Measure performance
print("Linear Regressor performance:")
print("Mean absolute error =", round(sm.mean_absolute_error(y_test, y_test_pred), 2))
print("Mean squared error =", round(sm.mean_squared_error(y_test, y_test_pred), 2))
print("Median absolute error =", round(sm.median_absolute_error(y_test, y_test_pred), 2))
print("Explained variance score =", round(sm.explained_variance_score(y_test, y_test_pred), 2))
print("R2 score =", round(sm.r2_score(y_test, y_test_pred), 2))
```

```
Linear Regressor performance:
Mean absolute error = 3.58
Mean squared error = 20.31
Median absolute error = 2.99
Explained variance score = 0.86
R2 score = 0.86
```

9. Create a polynomial regressor of degree 10. Train the regressor on the training dataset. Let's take a sample datapoint and see how to perform prediction. The first step is to transform it into a polynomial:

```
In [78]: # Polynomial regression
polynomial = PolynomialFeatures(degree=10)
X_train_transformed = polynomial.fit_transform(X_train)
datapoint = [[7.75, 6.35, 5.56]]
poly_datapoint = polynomial.fit_transform(datapoint)
```



If you look closely, this datapoint is very close to the datapoint on line 11 in our data file, which is [7.66, 6.29, 5.66]. So, a good regressor should predict an output that's close to 41.35.

Create a linear regressor object and perform the polynomial fit. Perform the prediction using both linear and polynomial regressors to see the difference:

```
In [79]: poly_linear_model = linear_model.LinearRegression()
poly_linear_model.fit(X_train_transformed, y_train)
print("\nLinear regression:\n", linear_regressor.predict(datapoint))
print("\nPolynomial regression:\n", poly_linear_model.predict(poly_datapoint))
```

```
Linear regression:
[36.05286276]
```

```
Polynomial regression:
[41.45537002]
```

As you can see, the linear regression is 36.05; the polynomial regressor is closer to 41.35 so the polynomial regression model was able to make a better prediction.



3.9.3 Estimating housing prices using a Support Vector Regressor

Let's see how to use the SVM concept to build a regressor to estimate housing prices.

We will use the dataset available in sklearn where each datapoint is defined by 13 attributes.

Our goal is to estimate housing prices based on these attributes.

1. Create a new Python file and import the following packages:

```
In [80]: import numpy as np
from sklearn import datasets
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error, explained_variance_score
from sklearn.utils import shuffle
```

2. Load the housing dataset:

```
In [81]: # Load Boston housing dataset
# Provided by data file boston_house_prices.csv
import pandas as pd

# Load the dataset
data = pd.read_csv('boston_house_prices.csv')

data_target = np.array(data.MEDV)
data_feature = np.array(data.iloc[:, :-1])

# Display the first few rows of the dataset
# print(data.head())

print(data_target.shape)
print(data_feature.shape)
```

```
(506,)
(506, 13)
```

3. Let's shuffle the data so that we don't bias our analysis:

```
In [82]: # Shuffle the data
X, y = shuffle(data_feature, data_target, random_state=7)
```

4. Check X and y to make sure everything is okay:

In [83]: X

```
Out[83]: array([[3.8497e+00, 0.0000e+00, 1.8100e+01, ..., 2.0200e+01, 3.9134e+02,
                1.3270e+01],
                [3.0410e-02, 0.0000e+00, 5.1900e+00, ..., 2.0200e+01, 3.9481e+02,
                1.0560e+01],
                [2.4103e-01, 0.0000e+00, 7.3800e+00, ..., 1.9600e+01, 3.9690e+02,
                1.2790e+01],
                ...,
                [8.4054e-01, 0.0000e+00, 8.1400e+00, ..., 2.1000e+01, 3.0342e+02,
                1.6510e+01],
                [4.0110e-02, 8.0000e+01, 1.5200e+00, ..., 1.2600e+01, 3.9690e+02,
                4.0800e+00],
                [6.6640e-02, 0.0000e+00, 4.0500e+00, ..., 1.6600e+01, 3.9096e+02,
                5.3300e+00]])
```

```
In [84]: y
```

```
Out[84]: array([21.7, 18.5, 22.2, 20.4,  8.8,  8.3, 21.4, 23.6, 23.5, 19. , 10.2,
 31.6, 18.4, 19.4, 44. , 23.2, 22. , 33.1, 13.2, 21.7, 16.7, 23.9,
 42.3, 18.5, 14.6,  7.5, 33.4, 22. , 28.1, 24.8, 19.9, 24.8, 28.7,
 14.1, 43.5, 20.4, 18.3, 13.4, 21. , 22. , 22.6, 13.9, 13.3, 26.5,
 20. ,  9.5, 21.6, 22.6, 15.6, 23.1, 27.1, 25.2, 15. , 21.9,  9.6,
 22.7, 18.3, 21.4, 20.5, 21.1, 24.5, 31. , 18.7, 18.7, 22.7, 50. ,
 16.7, 23. , 20.4, 22.8, 24.8, 10.5, 18.2, 31.5, 23.8, 19.6, 12.7,
 15.2, 31.2, 14.5, 17.8, 25. , 23. , 50. , 23.7, 25.1, 24.7, 16.8,
 34.7, 23.3, 50. , 46.7, 43.1, 24.4, 24.1,  6.3, 26.4, 16.3, 17.2,
 28. , 15.2, 16.6, 20.6, 16.4, 22. , 17.2, 13.5, 23.3, 26.2, 19.7,
 13.1, 23.9, 18.2, 19.3,  7. , 18.5, 13.8, 33.4, 12.6, 19.6, 20.8,
 25. , 21.7, 43.8, 13.4, 34.9, 24.5, 26.6, 36.5, 31.1, 22.6, 20.3,
 18.4, 28.7, 23.1, 17.6, 35.4, 31.6,  8.5, 22.2, 10.4, 22.8, 23.9,
 22.2, 32.9, 15.6, 29.8, 14.3, 28.4, 21.4, 23.1, 10.4, 20.1, 14.4,
 14.9, 10.2, 20.8, 50. , 23.3, 25. , 23.7, 11.9, 23.9, 20.6, 23.8,
 32. ,  8.3, 22.5, 37.2,  8.4, 23.4, 13.1, 17.8, 50. , 29.1, 19.4,
 11.7, 30.1, 27.5, 26.7, 33.2, 33.1, 17.8, 19.9, 50. , 50. , 20. ,
 22.5, 21. ,  7. ,  8.8, 19.8, 23.1, 23.4, 32.4, 20.1, 22.4, 42.8,
 23.1, 16.1, 22. , 23.6, 10.9, 14. , 29.1, 18.6, 36. , 22.6, 17.4,
 24.3, 41.3, 24.2, 18.4, 45.4, 21.2, 50. , 27.5, 16.5, 22.9, 18.6,
 20.1, 41.7, 20.2, 24.6, 10.5, 13.6, 35.4, 32.2, 13.4, 25. , 13.5,
 36.2, 26.4, 21.5, 22.5, 23.3, 27.9, 50. , 28.6, 38.7, 24. , 12.5,
 16.1, 17. , 20.5, 18.7, 17.1, 32.7, 19.1, 19.8, 21.2, 20.6, 16. ,
 19.5, 19.9, 21.4, 14.6, 12.7, 21.2, 14.2, 35.2, 27.5, 15.6, 23.1,
 14.5, 16.8, 15.7, 17.5, 13.6, 29.9, 11.7, 13.3, 30.1, 24.1, 25. ,
 20.9, 20. , 27. , 18.9, 11.8, 30.1, 16.2, 10.9, 18.9, 20.2, 23.2,
 31.7, 21.7, 18. , 15.6, 20.1, 27.1, 21.9, 50. , 20.7, 16.5, 13.1,
 14.8, 30.7, 33. , 20.9, 19.1, 19.4, 14.4, 24.4, 23.2, 21.6, 20.7,
 22.8, 23. , 22.2, 28.4, 19.5,  8.1, 17.8, 21.2, 19.3, 50. ,  7.2,
 19.2, 22.3, 20.8, 15.2, 25. ,  7.2, 11.5, 24.3, 20.3, 50. , 20.6,
 34.9, 12.7, 20.3, 19.1, 14.3, 28.2, 17.7, 24.3, 26.6, 50. , 48.8,
 17.1, 17.9, 20. , 15.1, 15. , 22.2, 11.9, 37.9, 19.5, 13. , 19.4,
 11.3, 28.7, 17.5, 29. ,  8.5, 23.1, 19.5, 15.6,  5. , 37.6, 23.7,
 37. , 15.4, 23.8, 46. , 21. , 22.3, 39.8, 27.5, 30.3, 20. , 18.1,
 29.8, 14.1, 34.6, 13.8, 50. ,  7.4, 17.4, 22. , 21.1, 19.6, 30.8,
 25. , 24.7, 18.9, 12. , 29. , 14.9, 36.1, 18.9, 16.2, 20.3, 33.2,
 18.8, 13.4, 21.7, 18.5, 19.4, 13.8, 12.8, 20.4, 14.9, 21.7, 24.4,
 16.6, 48.5, 11.8, 19.3, 19.6, 12.1, 22.9, 19.2, 21.8, 22.6, 31.5,
 19.7, 20.1, 17.3, 13.1, 13.3, 36.2, 17.2, 18.2, 23. , 10.2, 22.4,
  8.4, 17.8, 15.4, 44.8, 20.5, 13.8, 19.6, 25. , 19.8, 37.3, 48.3,
 15. , 35.1, 21.7, 29.6, 22.9, 23.7, 26.6,  7.2, 13.8,  5.6, 36.4,
 50. , 20.6, 19. , 21.8, 50. , 23.9, 17.1, 33.8, 24.7, 24.1, 19.9,
 28.5, 14.1, 34.9, 24.8, 18.8,  8.7, 32.5, 15.3, 19.4, 24.6, 30.5,
 21.2, 10.8, 32. , 22.9, 24.4, 22.1, 21.4, 24. , 11. , 23.2, 12.3,
 16.1, 17.4, 19.1, 23.8,  9.7, 21.5, 25.3, 21.9, 17.5, 22.8, 14.5,
  5. , 29.6, 24.5, 19.3, 27.9, 19.3, 22. , 20.6, 13.9, 33.3, 29.4])
```

5. Split the dataset into training and testing in an 80/20 format:

```
In [85]: # Split the data into training and testing datasets
num_training = int(0.8 * len(X))
X_train, y_train = X[:num_training], y[:num_training]
X_test, y_test = X[num_training:], y[num_training:]
```

6. Create and train the Support Vector Regressor using a linear kernel. The C parameter represents the penalty for training error. If you increase the value of C, the model will

fine-tune it more to fit the training data. But this might lead to overfitting and cause it to lose its generality. The epsilon parameter specifies a threshold; there is no penalty for training error if the predicted value is within this distance from the actual value:

```
In [86]: # Create Support Vector Regression model
sv_regressor = SVR(kernel='linear', C=1.0, epsilon=0.1)

# Train Support Vector Regressor
sv_regressor.fit(X_train, y_train)
```

```
Out[86]: SVR
SVR(kernel='linear')
```

7. Evaluate the performance of the regressor and print the metrics:

```
In [87]: # Evaluate performance of Support Vector Regressor
y_test_pred = sv_regressor.predict(X_test)
mse = mean_squared_error(y_test, y_test_pred)
evs = explained_variance_score(y_test, y_test_pred)
print("\n#### Performance ####")
print("Mean squared error =", round(mse, 2))
print("Explained variance score =", round(evs, 2))
```

```
#### Performance ####
Mean squared error = 15.38
Explained variance score = 0.82
```

8. Let's take a test datapoint and perform prediction:

```
In [88]: # Test the regressor on test datapoint
test_data = [3.7, 0, 18.4, 1, 0.87, 5.95, 91, 2.5052, 26, 666, 20.2, 351.34,
print("\nPredicted price:", sv_regressor.predict([test_data])[0])
```

Predicted price: 18.521780101664802



Workshop 3.9 : SVM Regression Analysis with Breast Cancer Dataset

1. By using the built-in Breast Cancer Dataset introduced at the beginning of this workshop, test the performance of system by using the newly learnt SVM Regression Analysis.
2. Compare the results with the previous test we have done. What is/are your findings?

Workshop 3.9 Answer

In [89]: `## Workshop 3.9 Coding Solution ##`

3.10 Summary

In this workshop, we learned supervised learning with classification and regression for prediction.

We discussed the data classification problem and how to solve it. We understood how to preprocess data using various methods. We also learned about label encoding and how to build a label encoder.

We discussed logistic regression and built a logistic regression classifier. We understood what a Naïve Bayes classifier is and learned how to build one.

We also learned how to build a confusion matrix. We discussed Support Vector Machines and understood how to build a classifier based on that. We learned about regression and understood how to use linear and polynomial regression for single-and multivariable data. We then used a Support Vector Regressor to estimate housing prices using input attributes.

In the next workshop, we will learn about predictive analytics and how to build a predictive engine using ensemble learning

In []: